Preventing Double Booking in Campus Event and Venue Management Systems: A Literature Survey

Gaurav Jadhav, Atharva Desai, Aditya Patil, Varad Rajboinwad, Tushar Deshpande, Chaitali Shahapurkar, Tanishka Patil, Srushti Mane, Sarvesh Khaladkar, Shravani Khandzode, Omkar Shinde, Gayatri Muttepawar

Department of Information Technology
Marathwada Mitra Mandal's College of Engineering
Pune, India

***Abstract*—Many colleges still handle venue requests for student events on paper or over email, and two clubs can end up holding the same hall at the same time. Recent web systems move this work online, yet a booking system can still double book when two requests pass the availability check at nearly the same moment. This survey reviews published work from four angles: campus event and venue systems, concurrency control in reservation systems, scheduling with time intervals, and access control with audit logging. Ten application papers are compared on approval workflow, conflict handling and reported concurrency testing. The campus papers describe approval and notification well, but the material we could read reports no test that sends simultaneous requests to an approval step. The papers that do test concurrency come from other settings, such as billiard tables and bus seats, and leave out multi-step approval and buffer time between events. We then discuss CampusOS, a platform our team built for one engineering college, as one attempt to close this gap using a row lock, a database exclusion constraint and an append-only log. In local integration tests, 20 simultaneous approvals for one slot produced exactly one winner. The system has not been deployed, and its load target remains unverified.**

***Index Terms*—audit logging, campus event management, concurrency control, double booking, exclusion constraint, role-based access control, venue reservation**

I. Introduction

Work on the smart campus covers a wide field. Muhamad *et al.* [1] pulled together 29 articles on campus features, the Internet of Things and cloud services, and Zhang *et al.* [2] reviewed campus technologies with a focus on the people who use them. Because these reviews look at the campus as a whole, they do not go into the details of a routine job that takes up real staff time every semester: deciding which club gets which hall, and when.

At our college this job started as a paper form. A club fills in a request, a faculty coordinator signs it, and nobody has a shared view of what has already been approved, so a clash can stay hidden until close to the event. Moving the form to a website fixes the shared view. It does not, by itself, fix the clash.

The reason is shown in Fig. 1(a). If a system checks that a slot is free and then writes the booking as a separate step, two requests can both pass the check before either one has written anything. Both then commit. Berenson *et al.* [3] call this class of anomaly write skew: each transaction reads the same data and writes a different row, so neither overwrites the other and nothing looks wrong to the database. The error appears only under simultaneous use, which is why a demonstration with one user at a time will not reveal it.

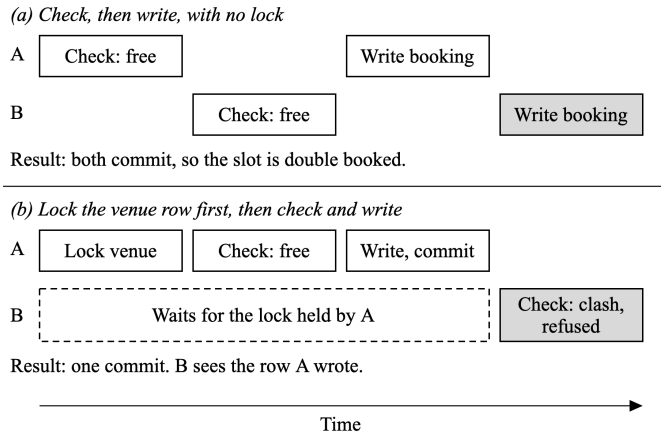


Fig. 1. Two requests for the same slot. (a) With separate check and write steps, both pass the check and both commit. (b) With a lock on the venue row, the second request waits, then sees the first booking and is refused.

This survey asks a narrow question: what does published work offer for keeping campus bookings free of double booking, and how well has it been tested? It makes three contributions. First, it compares ten application papers on features and on the evidence they report. Second, it links those papers to the older database and security literature that explains why certain designs hold up. Third, it lists the gaps found and shows how one working system responds to them, including what that system has still not proved.

Section II gives the survey method. Sections III to VI review the literature by theme. Section VII gives a short note on event recommendation. Section VIII compares the work and states the gaps. Section IX discusses CampusOS, and Section X concludes.

II. Survey Method

This is a narrative survey and not a systematic review, so we do not report screening counts. Application papers were found through OpenAlex and Crossref using search strings such as "college event management system web application," "venue booking system university conflict," "classroom reservation system web based" and "double booking concurrency reservation system database locking." We kept papers from 2016 onward that described a working reservation or event system. Older database, security and scheduling papers were added because later work rests on them.

Each bibliographic record was checked against Crossref by its DOI, apart from the online documents and the USENIX

paper, which were checked at their publisher sites. Full text could be obtained for three application papers [4]–[6]. We could not obtain the full text of the other seven, so they are known to us through their abstracts. We therefore write “not reported” and not “absent” for those seven, and Table I marks the basis of each entry. We did not search IEEE Xplore or Scopus directly, so relevant work may have been missed.

III. Campus Event and Venue Systems

A. Event Management

Jeny *et al.* [7] describe a web application where organisers publish college events, pick the date, location and equipment, and receive alerts when people view or register for an event. Kumar *et al.* [8] build a club management system on the MERN stack, with scheduling, login, participation tracking and live notifications. Both are concerned with running the event once it exists. Their abstracts do not discuss what happens when two organisers want the same place at the same time.

B. Venue and Classroom Booking

Selva Heera [9] uses React, Node.js, Express and SQL so that faculty can see availability in real time while an administrator approves or rejects requests from a dashboard, with email updates. EduNexus by Antony *et al.* [10] follows a similar plan on Node.js, Express and MongoDB, with roles for administrators and staff, a calendar view and reports. Simangunsong *et al.* [11] built their GoClass system with Scrum and added room suggestions based on availability and capacity, along with a dashboard of room use.

Two of these papers could be read in full. Ajith *et al.* [4] keep bookings in Firestore and use Firebase Authentication. Faculty and administrators get priority while students go through approval, and the system offers another slot or venue when a clash is found. The authors report booking times of 3 to 5 seconds and say the system prevented 99% of double bookings. The paper does not explain how that figure was measured. It also shows the difficulty with this measure, since a hall that is double booked even once in a hundred requests still leaves two events at one door.

Rodiyah *et al.* [5] built a reservation system at Esa Unggul University with CodeIgniter 4 and MySQL. Their security work is thorough for a paper of this kind, covering session login, input validation, CSRF protection and Cloudflare. Testing used a gray-box method and trials with students and staff. Against the manual process, they report that processing time fell from one or two days to under five minutes, and they rate the risk of a schedule conflict as low with system validation. Simultaneous requests are not part of their test plan.

C. What These Papers Leave Open

As a group, these papers make a strong case for moving bookings online and show that a small team can build such a system on common web stacks. They treat a conflict mainly as a question of what the user can see or what a form will accept. Whether the check still holds when requests overlap in time is the question taken up next.

IV. Concurrency Control

A. Foundations

Eswaran *et al.* [12] showed that consistency needs transactions to follow two-phase locking, and they also described phantoms, where a row that does not exist yet can still break a rule. This fits the booking case closely. The row that would cause a clash is one that another request has not yet inserted, so there is nothing to lock on the bookings table. Their proposed remedy, predicate locks, was hard to build efficiently, and practical systems look for other ways to get the same effect.

Kung and Robinson [13] offered the opposite approach. Optimistic control lets transactions run without locks and validates them at commit, restarting those that conflict. It suits work where conflicts are rare. Berenson *et al.* [3] later showed that the ANSI isolation levels were loosely defined and that snapshot isolation allows write skew. Ports and Grittner [14] closed that hole for PostgreSQL with serializable snapshot isolation, which tracks dangerous patterns of dependency and aborts a transaction when needed. The application then has to be ready to retry.

B. Reservation Systems That Test Concurrency

Altangerel *et al.* [15] study course selection, where many students compete for a limited number of seats in a centralised database. Shadrack and Andrew [16] present D-Mash, a distributed bus seat booking system, and report that the chance of double booking was brought down to zero. Hermawan and Suharso [6] give the clearest experiment we found. They ran a billiard table reservation system through 10 trials with 2 to 11 concurrent users. Before they added Firebase transactions every simultaneous attempt ended in double booking, and afterwards only one request was accepted in each trial. They also state plainly what they did not test: load beyond 11 users, real operating conditions, and latency, retry rate or throughput. Few papers in our set are this open about their limits.

The billiard and bus settings are simpler than a campus hall, however. A table or seat for a fixed slot gives one record to fight over. A venue request has a free choice of start and end time, so two requests can clash without naming the same slot. These systems also confirm a booking at once, while a campus booking may sit in a queue until someone approves it.

C. Design Options

The literature suggests three workable options for a relational system. Serializable isolation [14] needs little booking-specific code, though every writer must handle retries. A shared parent lock takes a row lock on the venue with SELECT ... FOR UPDATE before checking, so all writers for one venue queue up as in Fig. 1(b) [17]. This gets around the phantom problem because the venue row always exists. An exclusion constraint on a time range makes the database itself refuse overlapping rows [18], which behaves much like a predicate lock limited to one rule. It is independent of application code and so also catches a bug or a manual change.

Locks bring their own risk. Coffman *et al.* [19] set out the conditions for deadlock, and circular wait is the one most easily removed in application code, by always taking locks in a fixed order. PostgreSQL detects a deadlock and aborts one transaction [17], so a deadlock costs a failed request and not a wrong result, but it needs to be tested for all the same.

V. Scheduling With Time Intervals

Kolen *et al.* [20] survey interval scheduling, where jobs have fixed start and end times and must be placed on machines so that no two on one machine overlap. Treating venues as machines and events as jobs gives the booking problem a formal shape. Most of the surveyed results concern choosing a best set of jobs in advance. A college has a different need, since requests arrive one at a time and a person decides each one. Schaerf [21] covers automated timetabling for schools, courses and exams, where a whole timetable is built by search for the term. That work assumes the demand is known up front, so it does not transfer directly to ad hoc events, although a booking system does have to respect the classes it produces.

What does transfer is the overlap test. Two intervals clash when each one starts before the other ends. Halls also need time for setup and cleaning between events, so with a buffer B, a new booking clashes with an existing one when

$$(S_n < E_e + B) \text{ AND } (E_n + B > S_e) \quad (1)$$

where S and E are start and end times and the subscripts mark the new and existing bookings. Among the application papers we could read in full, none describes a buffer of this kind.

VI. Access Control, Audit and Web Architecture

A. Role-Based Access Control

Sandhu *et al.* [22] set out the family of role-based models, from core roles through hierarchies and constraints. The proposed NIST standard of Ferraiolo *et al.* [23] brought these ideas together into a shared reference model that includes separation of duty. A college fits this well, because people act as students, club leads and faculty coordinators, and the permissions belong to the job and not the person. Several of the surveyed systems name role-based control [4], [10], usually to separate administrators from everyone else.

B. Audit Logs

If a decision about a hall is later disputed, the college needs a record of who approved what. Schneier and Kelsey [24] show how to build logs that a later intruder cannot quietly alter or read, using chains of hashes and changing keys. That is a stronger threat model than a college needs on day one, but it sets the standard against which simpler logs should be judged. Ajith *et al.* [4] keep a booking history for audit. We did not find an application paper that says its log is protected against later edits.

C. Web Architecture and Sign-In

The stateless, resource-based style described by Fielding and Taylor [25] underlies the REST interfaces most of these systems use. JSON Web Tokens [26] carry the signed claims for stateless sign-in, while bcrypt [27] slows password guessing by making each hash deliberately costly. These are settled choices, and the surveyed systems differ more in which provider they use than in the ideas themselves.

VII. Event Recommendation

Adomavicius and Tuzhilin [28] group recommenders into content-based, collaborative and hybrid methods, and note the difficulty of new users and new items. That difficulty is serious for a single college, where history is thin and every event is new. The room suggestions of Simangunsong *et al.* [11] and the alternative slots of Ajith *et al.* [4] solve a different problem, namely finding a free venue. For suggesting events to students, a transparent scoring rule based on a few known signals is a reasonable start until enough data exists to train anything better.

VIII. Comparison and Research Gaps

Table I compares the ten application papers on what they report, with our own system added in the last row for reference. Entries reflect only the material we could read. They are not a judgement on what the authors may have built but did not describe.

Reading across Table I and the sections above, we see five gaps.

G1, approval under simultaneous use. None of the campus papers we read reports a test that sends simultaneous requests to a booking or approval step. The one figure offered [4] comes without a method.

G2, concurrency without workflow. The papers that do test concurrency [6], [16] work with fixed seats or slots, confirm at once, and have no approval step, so their results do not carry over directly to a hall that waits for a coordinator.

G3, one layer of defence. Where a mechanism is named, it sits at one level, either a check in the application or a transaction in the data store. The database literature [12], [18] supports adding a rule that the database enforces whatever the application does.

G4, buffer time. Setup and cleaning time between events is a plain practical need, yet the full-text papers do not model it.

G5, stated limits. Apart from Hermawan and Suharso [6], the papers we read in full say little about what remains unproved, such as behaviour under real load.

IX. CampusOS: A Response to the Gaps

CampusOS is a platform our team built for our own college using React 19, Express 5 and PostgreSQL 16, with 24 tables and five user roles. Since we are its authors, what follows is a report on our design and not an independent evaluation. Table II links each gap to the design and the evidence, drawn from a test run of 20 September 2026.

TABLE I
COMPARISON OF SURVEYED APPLICATION PAPERS

Ref.	Domain	Stack or data store	Approval step	Conflict handling	Concurrency evidence reported	Read
[7]	College events	Not stated	NR	NR	NR	A
[8]	Club events	MERN	NR	Scheduling module	NR	A
[9]	Campus venues	React, Node, Express, SQL	Administrator	Real-time availability view	NR	A
[4]	Campus venues	Firebase, Firestore	Students only; faculty have priority	Availability check; alternative suggested	99% of double bookings prevented; method not described	F
[10]	College facilities	Node, Express, MongoDB	Administrator	Calendar view	NR	A
[11]	Classrooms	Not stated	NR	Room suggested by availability and capacity	NR	A
[5]	Classrooms	CodeIgniter 4, MySQL	Administrator	Form validation by the system	None; functional gray-box testing only	F
[15]	Course seats	Centralised database	Not applicable	Concurrency control is the main topic	Details not in abstract	A
[6]	Billiard tables	Firebase transactions	None	Atomic transaction with retry	10 trials, 2 to 11 users; double booking fell from 100% to 0%	F
[16]	Bus seats	Distributed system	Not applicable	Distributed concurrency control	Double-booking probability reduced to zero	A
Ours	Campus venues and events	React, Express, PostgreSQL 16	Multi-tier	Venue row lock, buffer check, exclusion constraint	20 simultaneous approvals gave 1 winner; see Section IX	Own

NR = not reported in the material read. Read: F = full text, A = abstract only, Own = our system, for which we have full access.

TABLE II
GAPS, DESIGN RESPONSES AND EVIDENCE

Gap	Design in CampusOS	Evidence from tests
G1, G2	Requests and approvals lock the venue row with SELECT ... FOR UPDATE, then check and write in one transaction	20 simultaneous approvals for one slot: 1 succeeded, 19 refused
G3	Exclusion constraint on venue and time range for approved bookings	Overlapping approved row inserted directly in SQL was rejected
G4	Buffer of 0 to 120 minutes, default 15, applied by (1) inside the locked query	Covered by booking integration tests
G5	Limits are recorded in the test report and restated below	500-user target listed as unverified

The same pattern protects seats. When 20 students tried for the last seat at once, exactly one got it, and a database check stops the count from passing capacity. The audit table has a trigger that refuses updates and deletes, and a test confirms that such attempts fail. The full suite has 1,278 tests, all passing, with statement coverage of 98.71% on the backend and 94.17% on the frontend.

We also found faults of our own. Reading the code showed a real deadlock between two operations on clubs, the circular wait of Coffman *et al.* [19]. It was fixed by locking in a fixed order. A test of 15 addMember and 15 changeRole calls fails against the old code and now guards against its return.

These results have clear limits. The latency figures, with no p95 above 77 ms at 100 requests in flight, come from one laptop with a small data set and no network delay, so they are not a load test. The system has not been deployed and has had no trial with real users, so nothing here shows that it helps a real college. The exclusion constraint covers the raw time window only, which means the buffer depends on the application query being correct. The audit trigger stops ordinary changes, but a database administrator could disable it, so it falls well short of the guarantee in [24]. Sign-up for students of other colleges is also not built yet.

X. CONCLUSION

Published campus systems show that moving venue requests online is practical and useful. Their reported testing, however, seldom reaches the case where two requests arrive together. Papers that do test this case work on simpler reservations with no approval step. The older database literature already explains the fault and the remedies, so the gap lies in applying and testing them and not in new theory. Our experience with CampusOS suggests that a parent row lock with an exclusion constraint is a workable pairing for a small team, and that concurrency tests are worth writing, as one of ours now proves a fix for a real deadlock in our own code.

Further work should run a trial with real clubs and coordinators, carry out a proper load test on a deployed copy,

and add tamper-evident protection to the audit log. A wider search through IEEE Xplore and Scopus, with full-text access to the seven papers read here by abstract only, would make this comparison stronger.

Acknowledgment

The authors thank their project guide, Prof. Nishanthi Naidu, and the EDI Coordinator, Ms. P. R. Balage, of the Department of Information Technology, for their guidance during this work.

References

[1] W. Muhamad, N. B. Kurniawan, Suhardi, and S. Yazid, "Smart campus features, technologies, and applications: A systematic literature review," in *Proc. Int. Conf. Inf. Technol. Syst. Innov. (ICITSI)*, 2017, pp. 384–391, doi: 10.1109/ICITSI.2017.8267975.

[2] Y. Zhang, C. Yip, E. Lu, and Z. Y. Dong, "A systematic review on technologies and applications in smart campus: A human-centered case study," *IEEE Access*, vol. 10, pp. 16134–16149, 2022, doi: 10.1109/ACCESS.2022.3148735.

[3] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil, and P. O'Neil, "A critique of ANSI SQL isolation levels," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 1995, pp. 1–10, doi: 10.1145/223784.223785.

[4] R. Ajith, A. Sathvika, N. Srija, and P. Archana, "Campus venue booking system," *REST J. Data Anal. Artif. Intell.*, vol. 4, no. 1, pp. 115–124, Mar. 2025, doi: 10.46632/jdaai/4/1/13.

[5] S. Rodiyah *et al.*, "Implementation of a classroom reservation information system at Esa Unggul University," *J. Teknologi Open Source*, vol. 8, no. 2, pp. 453–466, Dec. 2025, doi: 10.36378/jtos.v8i2.4633.

[6] M. L. Hermawan and W. Suharso, "Implementation of concurrency control to prevent race condition in a web-based billiard table reservation system," *J. Inovtek Polbeng Seri Informatika*, vol. 10, no. 3, pp. 1971–1981, Nov. 2025, doi: 10.35314/vrnggg84.

[7] J. R. V. Jeny, P. Sadhana, B. J. Kumar, S. L. Abhishek, and T. S. Chander, "A web based-college event management system and notification sender," in *Proc. 4th Int. Conf. Inventive Res. Comput. Appl. (ICIRCA)*, 2022, pp. 1434–1438, doi: 10.1109/ICIRCA54612.2022.9985774.

[8] S. Kumar, S. Kumar, S. R. Chowdary, and E. Murali, "Club management system: A MERN stack-based web application for event management in colleges," *Int. J. Innov. Eng. Sci.*, vol. 10, no. 7, pp. 96–100, Jun. 2025, doi: 10.46335/IJIES.2025.10.7.20.

[9] Selva Heera R. J., "Campus venue booking," *Int. Res. J. Educ. Technol.*, vol. 6, no. 11, pp. 1008–1012, Nov. 2024, doi: 10.70127/irjedt.vol.7.issue03.1012.

[10] A. Antony, L. M. Thomas, M. K. Suresh, M. Sony, and R. Saritha R, "EduNexus: College facility booking platform," *Int. J. Eng. Sci. Technol.*, vol. 10, no. 1, pp. 36–38, 2026, doi: 10.29121/ijoest.v10.i1.2026.737.

[11] G. A. Simangunsong, M. R. Rusafni, G. J. P. Hakim, M. I. Nurfajri, A. Wicaksono, and G. P. Mindara, "Development of a web-based classroom reservation system using the Agile Scrum methodology," *J. Inform. Commun. Technol.*, vol. 7, no. 1, pp. 69–78, Jul. 2025, doi: 10.52661/jict.v7i1.409.

[12] K. P. Eswaran, J. N. Gray, R. A. Lorie, and I. L. Traiger, "The notions of consistency and predicate locks in a database system," *Commun. ACM*, vol. 19, no. 11, pp. 624–633, Nov. 1976, doi: 10.1145/360363.360369.

[13] H. T. Kung and J. T. Robinson, "On optimistic methods for concurrency control," *ACM Trans. Database Syst.*, vol. 6, no. 2, pp. 213–226, Jun. 1981, doi: 10.1145/319566.319567.

[14] D. R. K. Ports and K. Grittner, "Serializable snapshot isolation in PostgreSQL," *Proc. VLDB Endow.*, vol. 5, no. 12, pp. 1850–1861, Aug. 2012, doi: 10.14778/2367502.2367523.

[15] B. Altangerel, T. Purevsuren, and O.-E. Namsrai, "Managing concurrent course selection requests in a web-based university management system," in *Proc. IEEE 9th Int. Conf. Data Sci. Cyberspace (DSC)*, 2024, pp. 818–820, doi: 10.1109/DSC63484.2024.00123.

[16] Shadrack M. and Andrew M., "'D-Mash': A concurrent distributed booking system," *Int. J. Comput. Appl.*, vol. 139, no. 8, pp. 28–33, Apr. 2016, doi: 10.5120/ijca2016909243.

[17] PostgreSQL Global Development Group. "Explicit locking," in *PostgreSQL 16 Documentation*. Accessed: Nov. 2, 2026. [Online]. Available: https://www.postgresql.org/docs/16/explicit-locking.html

[18] PostgreSQL Global Development Group. "Range types," in *PostgreSQL 16 Documentation*. Accessed: Nov. 2, 2026. [Online]. Available: https://www.postgresql.org/docs/16/rangetypes.html

[19] E. G. Coffman, M. Elphick, and A. Shoshani, "System deadlocks," *ACM Comput. Surv.*, vol. 3, no. 2, pp. 67–78, Jun. 1971, doi: 10.1145/356586.356588.

[20] A. W. J. Kolen, J. K. Lenstra, C. H. Papadimitriou, and F. C. R. Spieksma, "Interval scheduling: A survey," *Nav. Res. Logist.*, vol. 54, no. 5, pp. 530–543, 2007, doi: 10.1002/nav.20231.

[21] A. Schaerf, "A survey of automated timetabling," *Artif. Intell. Rev.*, vol. 13, no. 2, pp. 87–127, Apr. 1999, doi: 10.1023/A:1006576209967.

[22] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *Computer*, vol. 29, no. 2, pp. 38–47, Feb. 1996, doi: 10.1109/2.485845.

[23] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST standard for role-based access control," *ACM Trans. Inf. Syst. Secur.*, vol. 4, no. 3, pp. 224–274, Aug. 2001, doi: 10.1145/501978.501980.

[24] B. Schneier and J. Kelsey, "Secure audit logs to support computer forensics," *ACM Trans. Inf. Syst. Secur.*, vol. 2, no. 2, pp. 159–176, May 1999, doi: 10.1145/317087.317089.

[25] R. T. Fielding and R. N. Taylor, "Principled design of the modern Web architecture," *ACM Trans. Internet Technol.*, vol. 2, no. 2, pp. 115–150, May 2002, doi: 10.1145/514183.514185.

[26] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF, RFC 7519, May 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7519

[27] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annu. Tech. Conf.*, 1999, pp. 81–91.

[28] G. Adomavicius and A. Tuzhilin, "Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions," *IEEE Trans. Knowl. Data Eng.*, vol. 17, no. 6, pp. 734–749, Jun. 2005, doi: 10.1109/TKDE.2005.99.